Analyzing an esp32 flash dump with ghidra

Olof Astrand · Follow

6 min read · Aug 3, 2020

As a third step I will use the flash loader to import the same binary, as in the previous story https://medium.com/@olof.astrand/enter-home-dragon-with-ghidra-3ed7ddf75935 . In order to get started you need to install ghidra and Xtensa processor support https://github.com/Ebiroll/ghidra-xtensa and the esp32 flash loader.

Digesting the esp32 flash image

Dump the flash of an esp32

```
esptool.py -p /dev/ttyUSB0 -b 460800 read_flash 0 0x400000
flash_contents.bin
```

Install the ghidra esp32 flash loader

Ebiroll/esp32_flash_loader

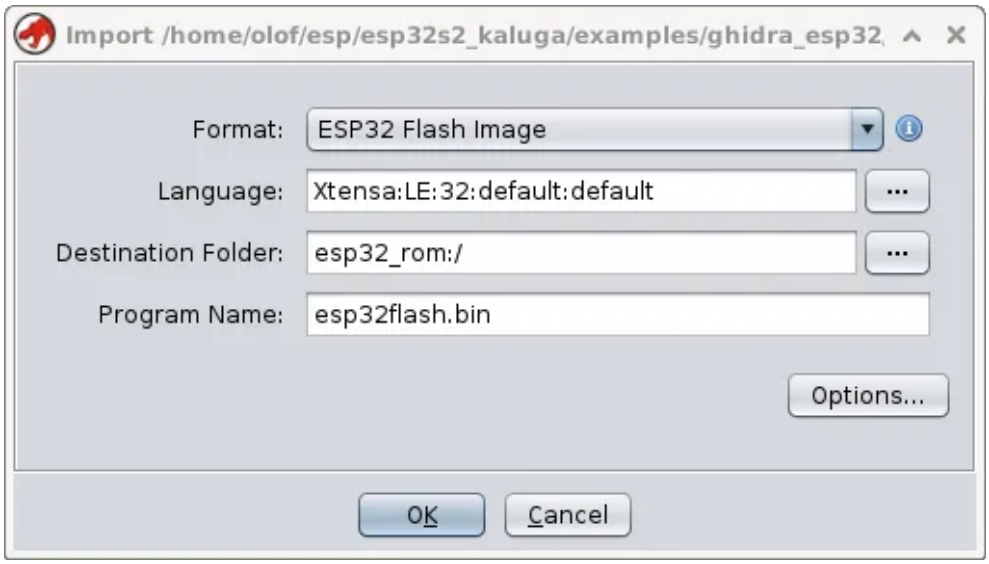
Ghidra Loader for ESP32 Flash Dumps GitHub is home to over 50 million developers working together to host and review...

github.com

Before import of the flash file

After installing the extension but before importing the flash file, consider downloading the esp32_rom.elf file and put it in the data directory. Put it next to the svd files. This way it will get loaded together with the svd and the flash file https://dl.espressif.com/dl/esp32_rom.elf

If you have installed the esp32 flash loader correctly, you will see this. If the flash contains several partitions, you can select the one to load with the options button.



Select the flash dump

As the flash file does not contain nearly as much as the corresponding elf file, we try to use as much information as possible that we know about the hardware and the esp32 rom.

Before jumping into the details you should re-consider reading about the esp32 memory model.

esp32 memory model

Memory used by the rom and the bootloader

The first 8KB (0x3FFA_E000–0x3FFA_FFFF) are used as a data memory for some of the ROM functions.

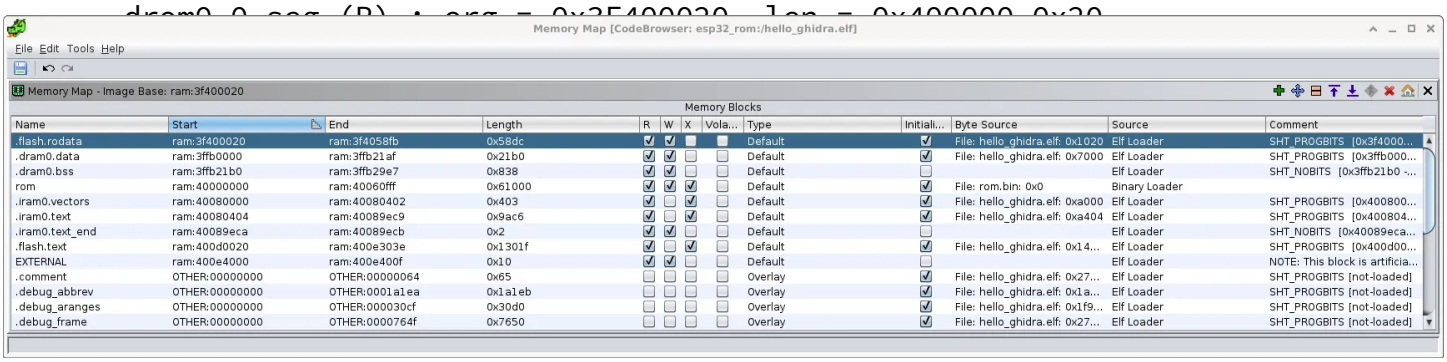
There are two regions within the heap (0x3FFE_0000–0x3FFE_0440–1088 bytes) and (0x3FFE_3F20–0x3FFE_4350=>1072 bytes) that are used by ROM code for its data. These regions are marked reserved and the heap allocator does not allocate memory from these regions.

Linker script

A short version of the linker script, build/esp-idf/esp32/esp32_out.ld

Generally speaking, addresses above 0x40000000 are on the instruction bus and below it, is on the data bus.

```
MEMORY
{
 iram0_2_seg (RX) : org = 0x400D0020, len = 0x330000-0x20
 dram0_0_seg (RW) : org = 0x3FFB0000 + 0x0,
                                       len = 0x2c200 - 0x0
  /* Flash mapped constant data */
```



hello.elf

```
}
```

```
  _static_data_end = _bss_end;
  /* Heap ends at top of dram0_0_seg */
  _heap_end = 0x40000000 - 0x0;
  _data_seg_org = ORIGIN(rtc_data_seg);
  /* The lines below define location alias for .rtc.data section based on
     Kconfig option.
     When the option is not defined then use slow memory segment
     else the data will be placed in fast memory segment */

  "Name","Start","End","Length"
  "DROM0_3f400020","ram:3f400020","ram:3f4058fb","0x58dc"
  "DRAM0_3ffb0000","ram:3ffb0000","ram:3ffb21af","0x21b0"
  "IRAM0_40080000","ram:40080000","ram:40080403","0x404"
  "IRAM0_40080404","ram:40080404","ram:40088553","0x8150"
  "IRAM0_400d0020","ram:400d0020","ram:400e303f","0x13020"
  "IRAM0_40088554","ram:40088554","ram:40089ecb","0x1978"


  iram0_2_seg
  (0x20 offset above is a convenience for the app binary image  generation.
  Flash cache has 64KB pages. The .bin file which is flashed to the chip has a
  0x18 byte file header, and each segment has a 0x08 byte segment header.)
```

After loading the binary dump

Window -> Memorymap

This shows where the ESP32 flash loader has loaded the flash

Bootlog

Compare this with the output from the bootloader when you run the program

```
I (4) esp_image: segment 0: paddr=0x00010020 vaddr=0x3f400020
size=0x058dc ( 22748) map
I (8) esp_image: segment 1: paddr=0x00015904 vaddr=0x3ffb0000
size=0x021b0 (  8624) load
I (9) esp_image: segment 2: paddr=0x00017abc vaddr=0x40080000
size=0x00404 (  1028) load
I (10) esp_image: segment 3: paddr=0x00017ec8 vaddr=0x40080404
size=0x08150 ( 33104) load
I (15) esp_image: segment 4: paddr=0x00020020 vaddr=0x400d0020
size=0x13020 ( 77856) map
I (27) esp_image: segment 5: paddr=0x00033048 vaddr=0x40088554
size=0x01978 (  6520) load
```

Heap Log

These are all the available data for allocations.

```
I (205) heap_init: Initializing. RAM available for dynamic allocation:
I (205) heap_init: At 3FFAE6E0 len 00001920 (6 KiB): DRAM
I (205) heap_init: At 3FFB29E8 len 0002D618 (181 KiB): DRAM
I (206) heap_init: At 3FFE0440 len 00003AE0 (14 KiB): D/IRAM
I (206) heap_init: At 3FFE4350 len 0001BCB0 (111 KiB): D/IRAM
I (207) heap_init: At 40089ECC len 00016134 (88 KiB): IRAM
I (207) cpu_start: Pro cpu start user code
```

Application output

When running the program, this is the output.

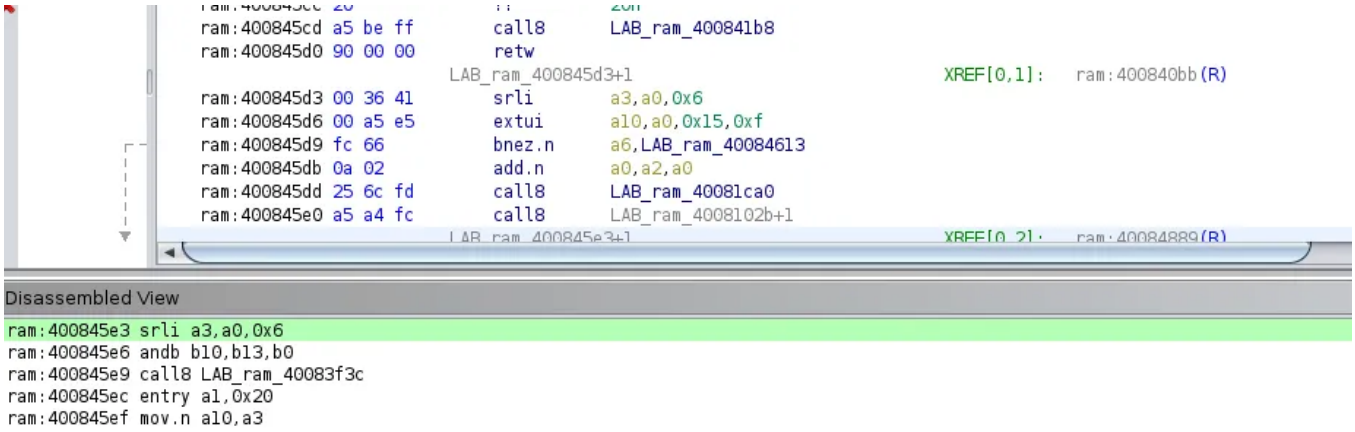
```
Hello Ghidra!
This is esp32 chip with 2 CPU cores, WiFi/BT/BLE, silicon revision 0,
2MB external flash
Free heap: 299616
my_int=4
i = 0
i = 1
i = 2
i = 3
?silly.my_int=14
ret=14
ret=9
Restarting now. 6
```

After importing you should only see one function named FUN_XX. This is the call_start_cpu0 function and the last function called will be start_cpu0_default(). You can have an elf compiled application open in order to identify the functions. Also try to identify ets_printf()

```
call_start_cpu0() {
...
uVar5 = func_0x40087a30(uVar7,uVar6,uVar5,uVar8,iVar3);
 (*(code *)PTR_ets_printf_ram_40080518)
(_LAB_ram_40080582+2,uVar5,_LAB_ram_40080500,uVar8,iVar3);
 start_cpu0_default();
}
```

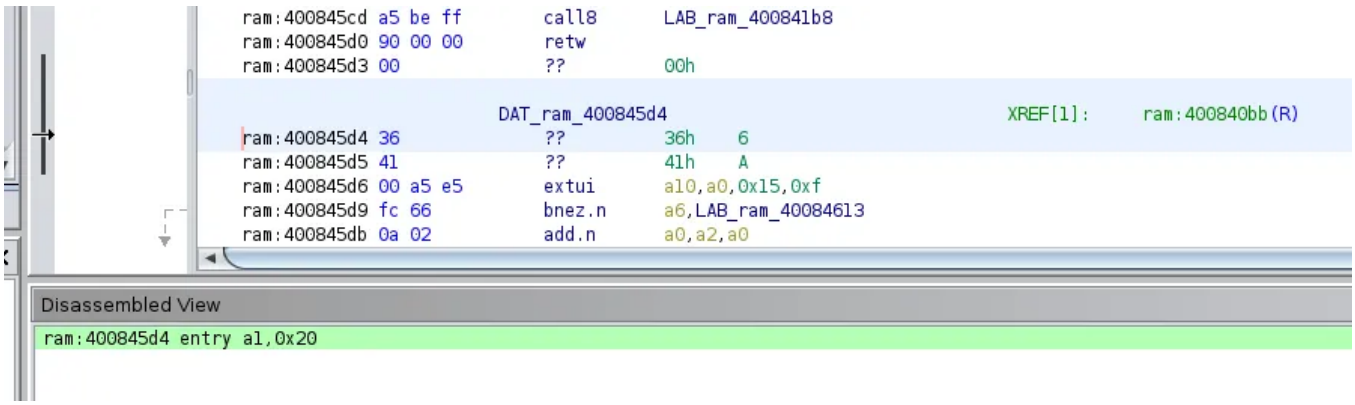
Manual corrections of ghidra labels

The LAB_ram + 1 is a dead giveaway that the functions is not correctly identified. The 00 is just code padding and was incorrectly identified as part of the next instruction.



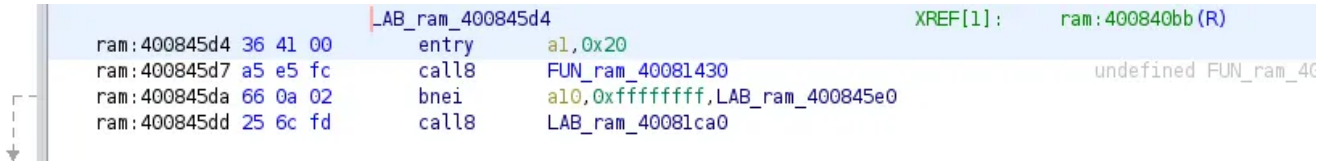
Faulty analysis

The way to solve this is to press C which will clear the disassembly. Also note the Disassembled view which identifies the entry instruction. You can identify this by the 0x36 byte which is easy to spot.



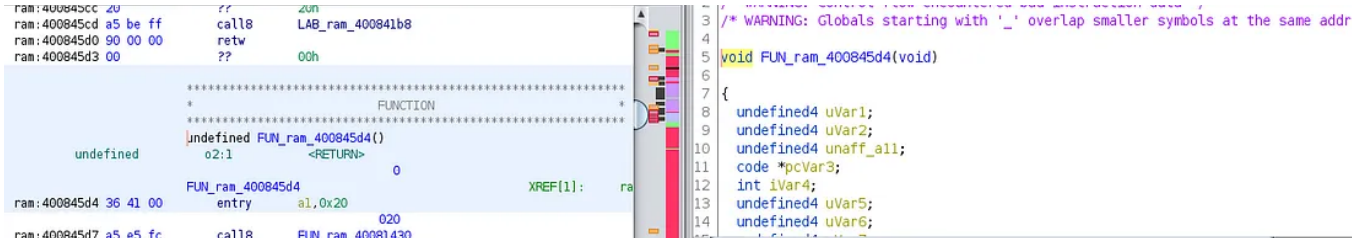
Identify the entry instruction

Repeat pressing C and D to get the instruction correctly.



Problem fixed

Finally mark the function with F



Marked as function

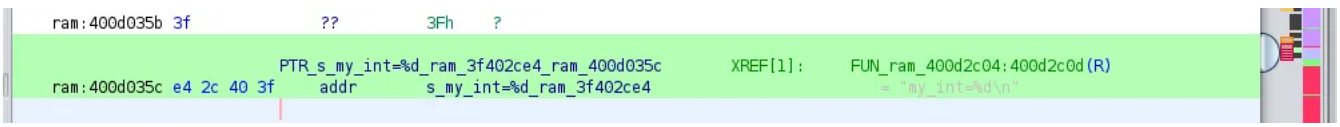
After identifying some more functions and data pointers, you can do auto analyze again. This will decrease the amount of Undefined memory space.

Legend

Clear some faultily identified instructions and mark it as data.

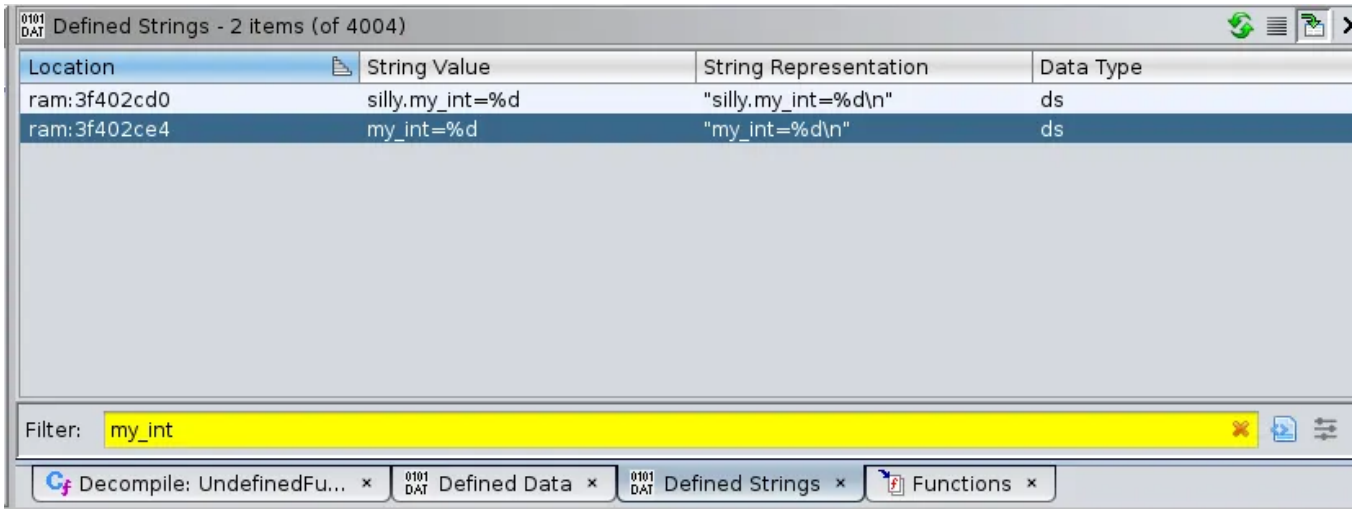
Before press of P

Press P , or right click data->pointer



Data pointer, PTR

Now we can click search for the string



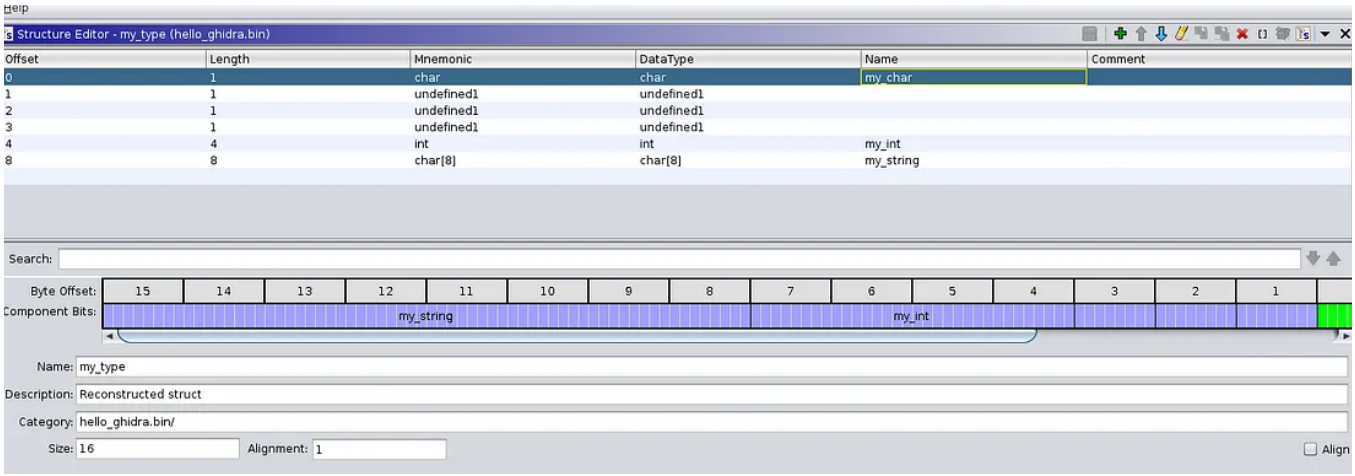
When clicking the reference we find our function.

```
undefined4 one_arg(int my_arg)
{
  int iVar1;
  undefined4 unaff_a13;
  undefined4 unaff_a14;
  undefined4 unaff_a15;
  undefined buffer [8];
  undefined auStack56 [4];
  undefined4 uStack52;

  uStack52 = 4;
  iVar1 = 0;
  while (iVar1 < my_arg) {
    buffer[iVar1] = 0x32;
    iVar1 = iVar1 + 1;
  }
  func_0x400e2adc(auStack56);

printf(_LAB_ram_400d0352+2,my_arg,buffer,unaff_a13,unaff_a14,unaff_a15)
;

printf(_DAT_ram_400d0358,uStack52,buffer,unaff_a13,unaff_a14,unaff_a15)
;
  return uStack52;
}
```

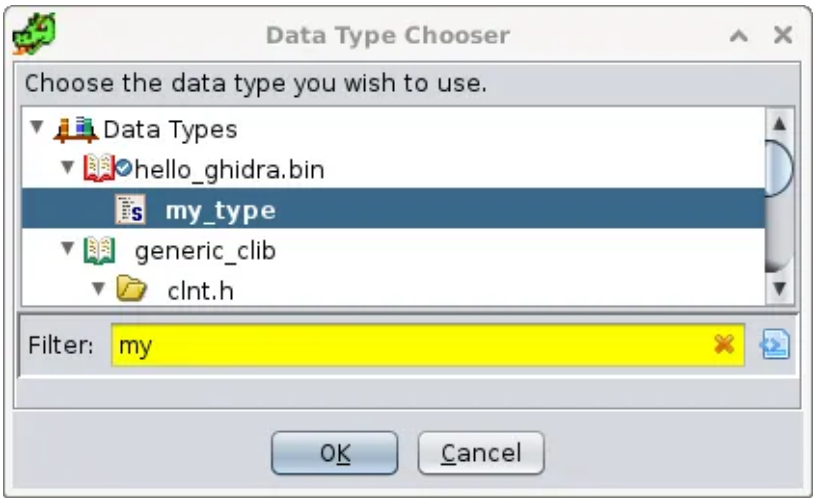
However we need to see this,

```
int one_arg(int my_arg){
    char buffer[8];
    int i;
    my_type silly;
    silly.my_int=4;
    for (i = 0; i < my_arg; ++i) {
        buffer[i]=0x32;
    }
    one_ptr_arg(&silly);
    printf("My_arg=%d,%s",my_arg,buffer);
    printf("silly.my_int=%d\n",silly.my_int);
    return (silly.my_int);
}
```

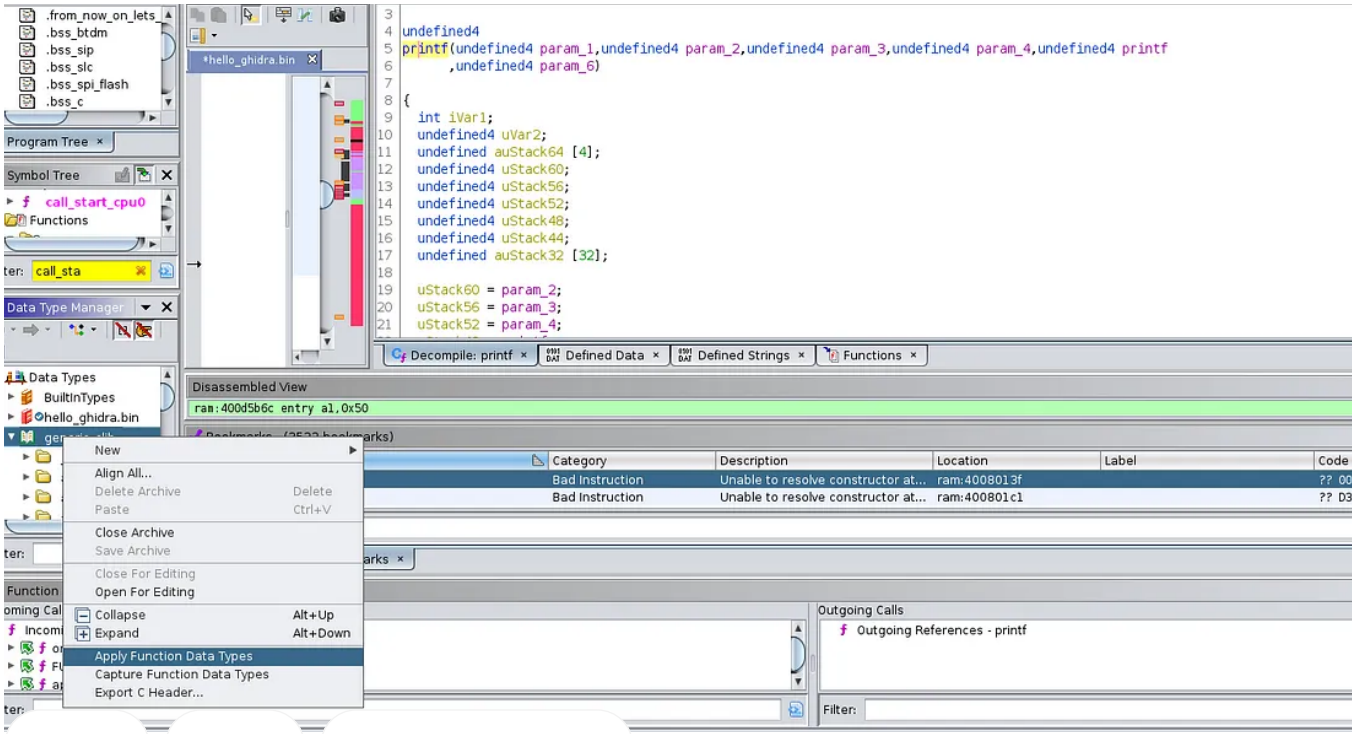
We could import the header file, but chose to add my_type manually.



Now we retype ustack56 Ctrl+L and use my_type



Also identify some functions and use the generic_clib and apply function data types.



Ghidra Esp32 Reverse Engineering

Now we have reconstructed the functions completely.

```
int one_arg(int my_arg)
{
  int i;
  ar buffer;
  _type silly;
```

Follow

```
    silly.my_int = 4;
    i = 0;
    while (i < my_arg) {
      (&buffer)[i] = '2';
      i = i + 1;
    }
    one_ptr_arg((int)&silly);
    printf(PTR_s_My_arg=%d,%s_ram_3f402cc0_ram_400d0354,my_arg,&buffer);
    printf(PTR_s_silly.my_int=%d_ram_3f402cd0_ram_400d0358,silly.my_int,&
  buffer);
    return silly.my_int;
  }
```

Written by Olof Astrand

51 Followers · 19 Following

Software Engineer.

Responses (1)

What are your thoughts?

Respond

pro-8c7c58871e68

Trevor Boultwood
Mar 17, 2021

Ive followed your guide, but im struggling with the installation of the esp32_rom.elf where should i place this?

Thank you

1 reply Reply

More from Olof Astrand

Olof Astrand

Reverse engineering of esp32 flash dumps with ghidra or IDA Pro

This story we take an flash dump from an esp32 and then use this to create and elf file, in order to recreate the original source code.

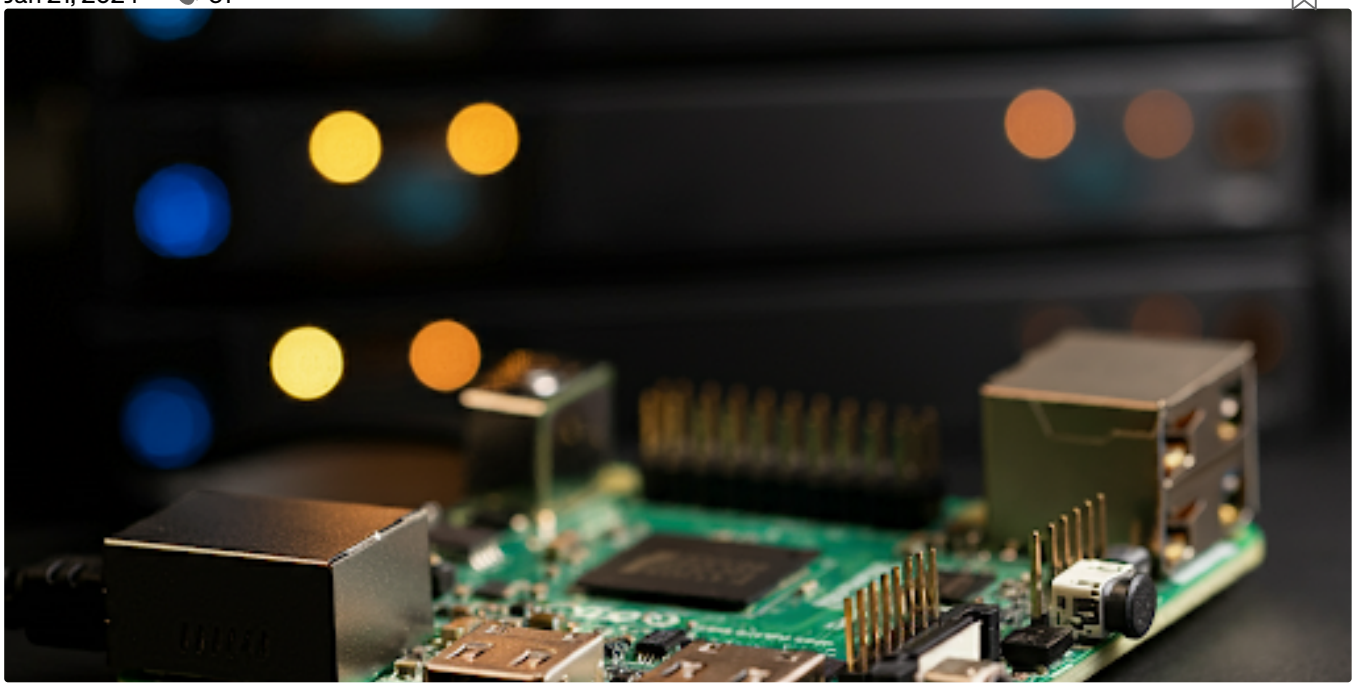
Apr 20, 2021 27

Olof Astrand

Running Ghidra Debugger IN-VM

Starting up ghidras debugger tool can be a bit confusing the first time you use it. Here I will go through how to do it first with a local...

Jan 21, 2024 51



Olof Astrand

Exploring QEMU and QNX on the Raspberry Pi 4: Using the Device Tree, Serial Ports and QEMU.

When I discovered that qemu has added support of the raspberry pi 4 (v9.0) I was really happy, but after testing I was really disapointed...

Sep 28, 2024 1

Olof Astrand

Hacking wireless sockets like a NOOB

Learn how to use the Universal Radio Hacker, software, in order to analyze the signals used to control a wireless socket device.

Apr 10, 2021 13

Recommended from Medium

See all from Olof Astrand

Harendra

How I Am Using a Lifetime 100% Free Server

Get a server with 24 GB RAM + 4 CPU + 200 GB Storage + Always Free

Oct 26, 2024 9K 146

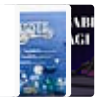
Jessica Stillman

Jeff Bezos Says the 1-Hour Rule Makes Him Smarter. New Neuroscience Says He's Right

Jeff Bezos's morning routine has long included the one-hour rule. New neuroscience says yours probably should too.

Oct 30, 2024 23K 647

Lists

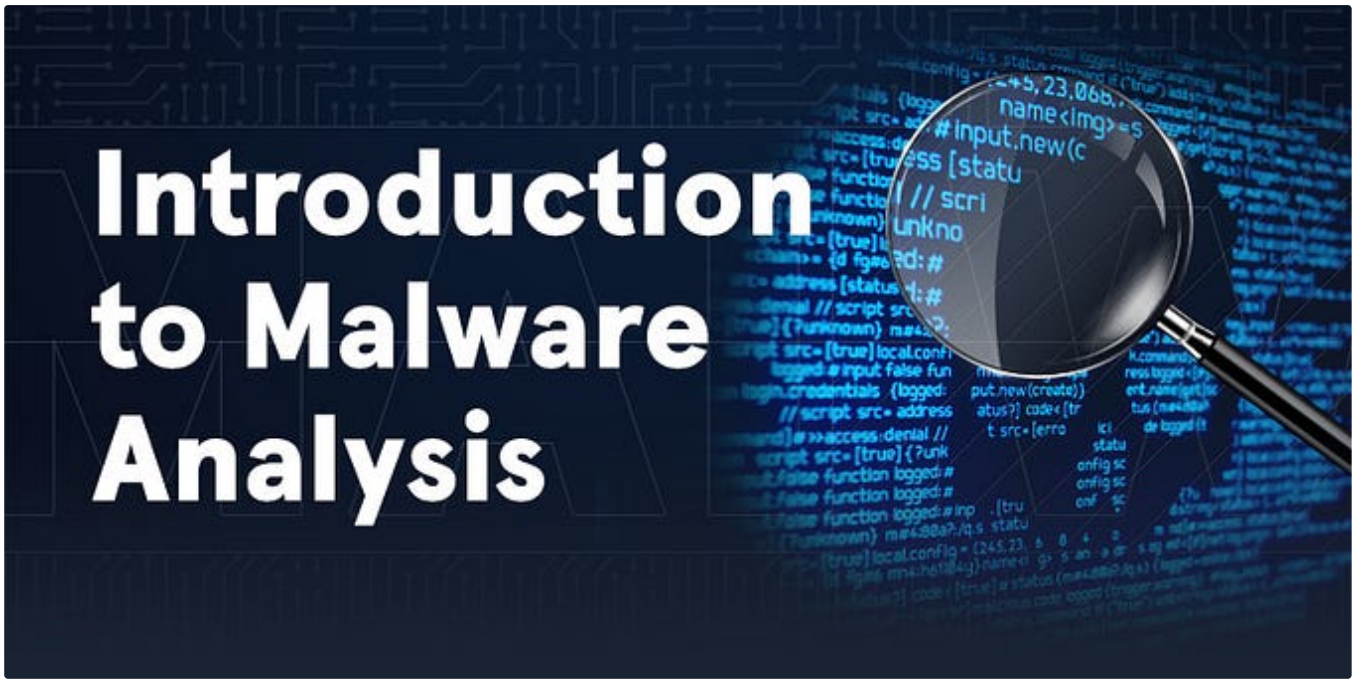


Staff picks
815 stories · 1626 saves

Stories to Help You Level-Up at Work
19 stories · 941 saves

Self-Improvement 101
20 stories · 3309 saves

Productivity 101
20 stories · 2786 saves



Eric H

Malware Analysis: Introductory

Chapter 1: Malware Analysis

Mario Bergeron

Tria Vitis Platforms — Adding support for ROS2

This project describes how to add support for ROS2 to the Tria Vitis Platforms.

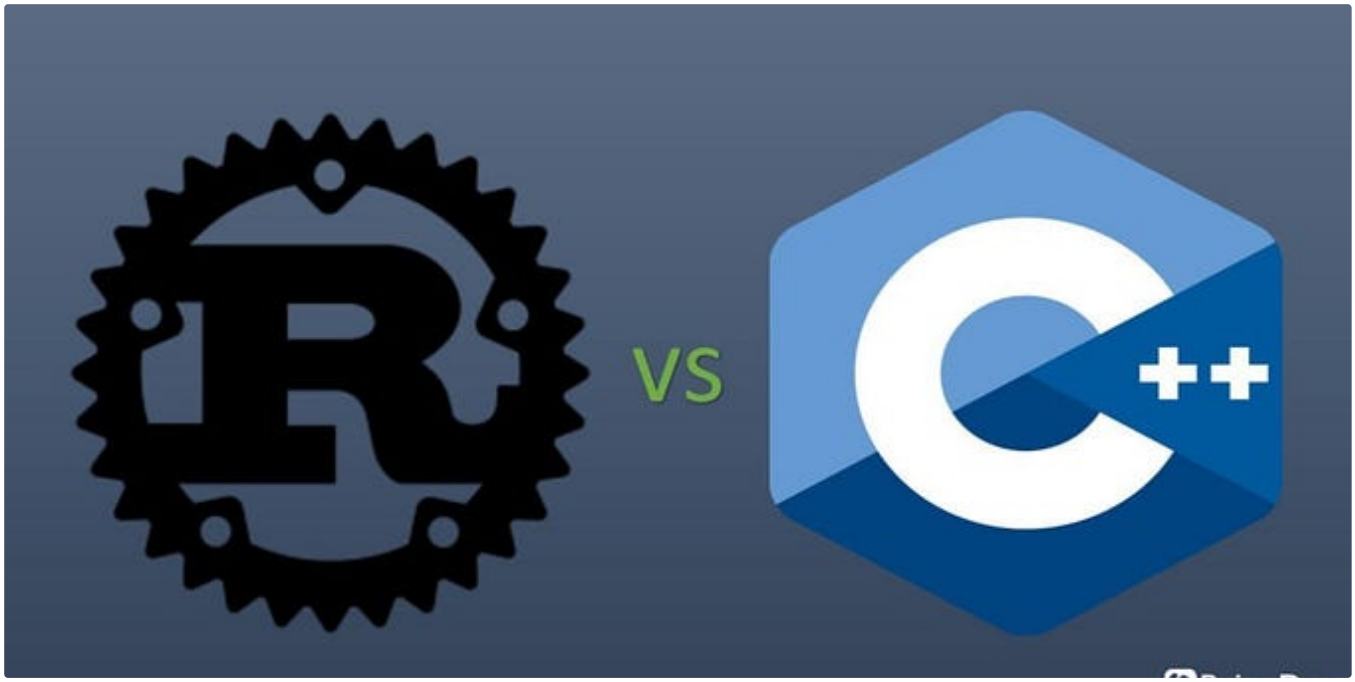
Nov 25, 2024

Olof Astrand

Exploring QEMU and QNX on the Raspberry Pi 4: Using the Device Tree, Serial Ports and QEMU.

When I discovered that qemu has added support of the raspberry pi 4 (v9.0) I was really happy, but after testing I was really disapointed...

Sep 28, 2024 1



RED TEAM | Malforge Group

Shadows in Rust: Crafting Advanced Windows Malware

“How Rust is Revolutionizing Malware Development and Evasion Techniques”

Sep 22, 2024 121 1

See more recommendations